

SYSTEM DESIGN INTERVIEW

Design a Web Crawler

Distributed crawler at Common Crawl / Googlebot scale

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · URL Frontier (Mercator)
- 06 · DNS & Politeness
- 07 · Fetcher Pool
- 08 · Content Parser & Link Extraction
- 09 · URL Deduplication
- 10 · Content Deduplication
- 11 · Storage Tiers
- 12 · Refresh & Recrawl Policy

- 13** · Scalability & Distribution
- 14** · Failure Modes & Recovery
- 15** · Design Trade-offs
- 16** · Interview Playbook

01

Problem Statement

Requirements and scope

Design a **distributed web crawler** like **Googlebot** or **Common Crawl**. The crawler discovers and downloads web pages, extracts links to feed back into the frontier, deduplicates content, respects `robots.txt` and per-host politeness, and stores the harvested HTML for downstream search-indexing and knowledge-extraction pipelines.

Clarifying Questions

Question	Answer
Pages to crawl?	10 billion unique URLs (web-scale corpus)
Refresh cycle?	30 days end-to-end (high-priority pages: 1 day)
Content types?	HTML primarily; sniff non-HTML and skip binaries
Politeness?	Obey robots.txt; max 1 req/sec per host
Output?	Raw HTML (S3) + extracted links + content fingerprints
Languages?	All; UTF-8 detection per page
JS rendering?	Phase 2: headless Chrome for SPA-heavy hosts
Duplicate handling?	URL dedup + near-dup content (SimHash)

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Seed	Accept seed URLs; bootstrap the frontier
Fetch	Download HTML over HTTP(S); follow redirects
Parse	Extract links + metadata + canonical tags
Enqueue	Push new URLs back into the frontier
Dedup	URL canonicalization + content near-dup detection
Politeness	Obey robots.txt; per-host rate limit
Store	Persist raw HTML and metadata for downstream pipelines
Refresh	Re-crawl pages on TTL; prioritise by importance

Non-Functional Requirements

Requirement	Target
Throughput	~3,860 fetches/sec sustained, ~8K peak
Storage	~750 TB/year compressed deduped
Freshness	30-day full refresh; 1-day for top-1% URLs
Politeness	≤ 1 req/sec/host; honour Crawl-Delay
Robustness	Survive zone outages; idempotent retries
Scalability	Linear scale-out: add fetcher pods for more QPS
Extensibility	Pluggable parsers (HTML, PDF, JSON-LD)

03

Capacity Estimation

The math

Throughput

- Pages to crawl: **10 billion**
- Refresh cycle: **30 days** $\Rightarrow$ 333M pages/day
- Sustained: $333\text{M} / 86,400 \text{ sec} \approx$ **3,860 fetches/sec**
- Peak (2 $\times$ sustained): **$\sim$ 8K fetches/sec**
- Each fetcher worker: $\sim$ 10 concurrent fetches @ $\sim$ 2 sec each $\rightarrow$ $\sim$ 5 fetches/sec/worker $\Rightarrow$ **$\sim$ 1,600 workers** sustained

Storage

- Avg page size: **100 KB** raw HTML
- Daily raw: $333\text{M} \times 100 \text{ KB} \approx$ **33 TB/day**
- Annual raw: $33 \text{ TB} \times 365 \approx$ **12 PB/year**
- Dedup ($\sim$ 30% identical content removed) $\Rightarrow$ $\sim$ 8.4 PB/year
- gzip compression ($\sim$ 5 $\times$) $\Rightarrow$ **$\sim$ 750 TB/year stored**
- URL set: 10B keys; Bloom filter @ $\sim$ 10 bits/key $\Rightarrow$ **$\sim$ 12 GB**
- Content fingerprint set: 10B SimHashes $\times$ 8 B $\Rightarrow$ **$\sim$ 80 GB**

DNS Pressure

- Without cache: $\sim$ 8K fetches/sec $\approx$ **$\sim$ 10K DNS qps** at peak
- With aggressive caching (TTL = host avg fetch interval): **$\sim$ 50 qps**
- Cache hit rate: $> 99\%$ (we re-hit the same hosts repeatedly)

KEY

Key Numbers

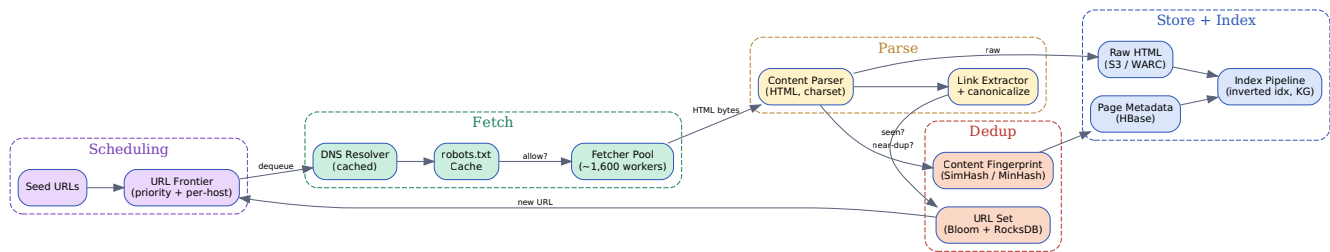
10B URLs $\cdot$ 30-day refresh $\Rightarrow$ **3,860 fetches/sec** sustained, **$\sim$ 8K peak** $\cdot$ 33 TB/day raw $\Rightarrow$ **$\sim$ 750 TB/year** after dedup & gzip $\cdot$ URL Bloom filter **$\sim$ 12 GB**.

04

High-Level Architecture

System overview

The crawler is a feedback loop: the **URL Frontier** feeds the **Fetcher Pool**, which downloads HTML; the **Content Parser** extracts links and emits them back into the frontier while the raw bytes flow into the **Content Store** and the indexing pipeline. Three out-of-band services keep the loop honest: **DNS**, **robots.txt cache**, and the **URL/Content Dedup** layer.



Core pipeline: frontier → DNS → fetch → parse, then a fan-out to link extraction (loop) and content storage / indexing (sink).

Component Responsibilities

- **URL Frontier:** prioritised, per-host queue feeding fetchers; the heart of the crawler
- **DNS Resolver:** async resolver with massive cache; ~99% hit rate
- **robots.txt Cache:** per-host fetched once per 24h; enforces Disallow + Crawl-Delay
- **Fetcher Pool:** stateless async HTTP workers; ~1,600 pods sustained
- **Content Parser:** charset detection, HTML → DOM, extract links + canonical tag
- **Dedup:** Bloom filter for URL membership; SimHash for near-dup content
- **Content Store:** S3 / WARC files for raw HTML; HBase for metadata index
- **Index Pipeline:** downstream MapReduce / Spark builds inverted index + KG

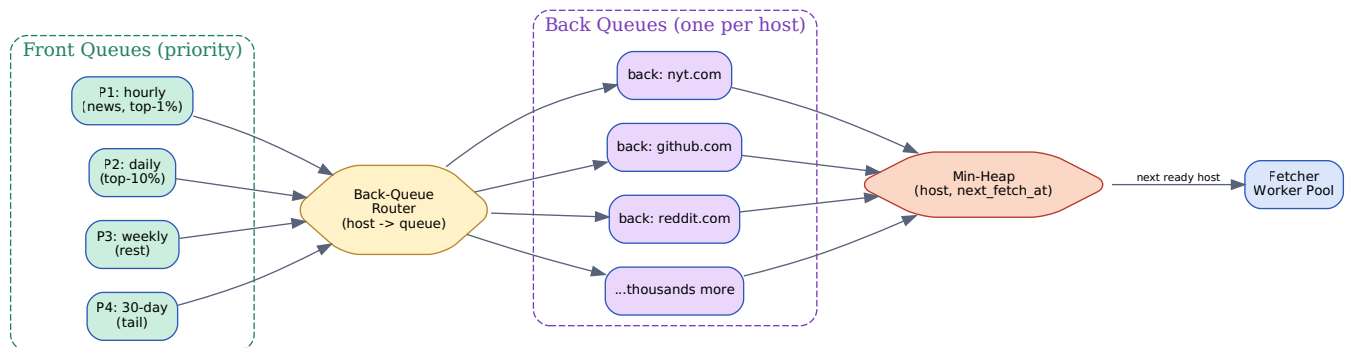
05

URL Frontier (Mercator)

Priority + politeness queue

Two-Level Queue

The frontier follows the **Mercator** design: **front queues** hold URLs sorted by priority (importance, freshness, refresh policy); **back queues** hold URLs grouped by host and feed the fetchers. A back-queue selector enforces politeness by ensuring each fetcher worker only ever pulls from one host at a time.



Mercator-style frontier: priority front queues feed per-host back queues; a heap-driven router picks the next host whose politeness window has elapsed.

Politeness Scheduler (pseudocode)

```

# Per-host min-heap keyed by next_fetch_at
# Invariant: a host is in the heap iff its back-queue is non-empty

def next_url():
    host, ready_at = heap.peek()
    if now() < ready_at:
        sleep(ready_at - now())
    heap.pop()
    url = back_queue[host].pop_front()
    crawl_delay = robots[host].crawl_delay or 1.0 # >=1 sec/host
    if back_queue[host]:
        heap.push((host, now() + crawl_delay))
    return url

def enqueue(url):
    host = canonical_host(url)
    was_empty = not back_queue[host]
    back_queue[host].push_back(url)
  
```

```
if was_empty:
    heap.push((host, now())) # immediately ready
```

Frontier Storage

- **In-memory:** hot back-queues fit in RAM (recent + active hosts)
- **On-disk overflow:** RocksDB / Kafka per partition for cold tail
- **Sharding:** partition by host hash so each frontier shard owns disjoint hosts
- **Persistence:** WAL on every dequeue/enqueue; recover state on crash

06

DNS & Politeness

Resolving and respecting hosts

DNS Resolver

- **Async resolver:** non-blocking (c-ares, getdns); thousands of in-flight lookups
- **Local cache:** per-fetcher LRU keyed by host; honour TTL but cap at 1 day
- **Shared cache:** Redis-backed cluster cache: $\sim 10\text{K hosts} \times \sim 64\text{ B} = \text{trivial}$
- **Negative cache:** cache NXDOMAIN for 60 sec to avoid hammering bad hosts
- **Effect:** $\sim 10\text{K qps} \rightarrow \sim 50\text{ qps}$ with cache (hit rate $>99\%$)

robots.txt

- **Fetch once per host per 24 h;** store parsed allow/disallow + Crawl-Delay
- **Match user-agent:** identify ourselves (e.g. MyCrawler/1.0) and obey wildcard fallbacks
- **Apply before fetch:** if disallowed, drop URL and emit a metric
- **Crawl-Delay:** overrides our default of 1 req/sec when host requests slower

Per-Host vs Per-IP Politeness

Strategy	Pros	Cons
Per-host	Simple; matches robots.txt scope	Co-hosted sites on same IP get hammered (e.g. shared hosting)
Per-IP	Friendly to shared hosting and CDNs	Need reverse-DNS; one big IP (Cloudflare) blocks unrelated sites
Hybrid	Default per-host; per-IP cap as upper bound	More state; reconcile when DNS changes; complexity

TIP

Recommendation

Start with **per-host** politeness (1 req/sec or Crawl-Delay). Layer on a **per-IP cap** (e.g., 10 req/sec/IP) only after you see complaints from shared-hosting providers.

07

Fetcher Pool

Async HTTP workers

Worker Design

- **Async I/O:** Python asyncio + aiohttp, or Go net/http with goroutines
- **Concurrency:** ~10 in-flight fetches per worker; bound by mem + per-host limit
- **Timeouts:** 5 sec connect, 30 sec total; abandon stragglers fast
- **Retry:** exponential backoff for 5xx / network errors; max 3 retries
- **Body size cap:** 10 MB; truncate larger pages
- **HTTPS:** reuse TLS sessions; ALPN HTTP/2 where supported

Sizing Math

- Average fetch wall time: ~2 sec (TCP + TLS + first-byte + read)
- Per-worker QPS: 10 concurrent / 2 sec = 5 fetches/sec
- Sustained workers: 3,860 / 5 = **~770**
- Headroom (HA + retry + tail): 2× ⇒ **~1,600 workers**
- Each worker pod ~256 MB ⇒ **~400 GB** total fetcher memory

Fetch Loop (pseudocode)

```
async def fetch_loop():
    while True:
        url = await frontier.next_url()
        host = canonical_host(url)
        ip = await dns.resolve(host)
        if not robots[host].allows(url):
            metrics.inc('robots_disallow')
            continue
        try:
            resp = await http.get(url, host_ip=ip,
                                timeout=30, max_bytes=10_000_000)
        except (Timeout, ConnError) as e:
            await retry_or_drop(url, e)
            continue
        if resp.status in (301, 302):
            await frontier.enqueue(canonicalize(resp.location))
            continue
```

```
if resp.status == 200 and is_html(resp):  
    await parser_queue.put((url, resp.body))
```

08

Content Parser & Link Extraction

From bytes to URLs

Parsing Stages

1. **Charset detection:** sniff UTF-8 / Latin-1 / declared meta
2. **HTML → DOM:** lxml or Cheerio; tolerant of malformed markup
3. **Canonical tag:** if <link rel=canonical>, prefer that URL
4. **Extract links:** all <a href>, plus src attrs as configured
5. **Normalize:** strip fragments, lowercase host, sort query params, drop session IDs
6. **Filter:** reject schemes other than http(s); reject blacklisted TLDs
7. **Emit:** push (url, source_url, anchor_text) tuples to the dedup layer

URL Canonicalization

```
def canonicalize(url):
    u = urlparse(url)
    scheme = u.scheme.lower()
    host = u.hostname.lower().rstrip('.')
    if (scheme, u.port) in [('http', 80), ('https', 443)]:
        netloc = host
    else:
        netloc = f'{host}:{u.port}' if u.port else host
    path = re.sub(r'\/+', '/', u.path) or '/'
    # remove tracking params; sort the rest
    params = [(k, v) for k, v in parse_qs(u.query)
               if k not in TRACKING_PARAMS]
    query = urlencode(sorted(params))
    # drop fragment entirely
    return urlunparse((scheme, netloc, path, '', query, ''))
```

Anchor-text Signal

- **Save anchor text:** downstream PageRank-style scoring uses it
- **Source → target graph:** store (src_url, dst_url, anchor) triples
- **Fan-out cap:** ≤ 1,000 outgoing links per page (defend against link bombs)

09

URL Deduplication

Have we seen this URL?

Two-Layer Approach

- **Bloom filter (in-memory)**: fast probabilistic 'definitely-new' test
- **RocksDB (on-disk)**: exact set; lookup on Bloom-positive to confirm
- **Why both**: Bloom lets us skip 99% of disk reads; RocksDB resolves the 1%

Bloom Filter Sizing

- Keys: **10 billion** URLs (target capacity)
- Bits/key: **~10** $\Rightarrow$ false-positive rate $\sim 1\%$
- Total bits: $10\text{B} \times 10 = 100\text{ Gbit} \Rightarrow \mathbf{\sim 12\text{ GB}}$
- Hash functions: **k = 7** (optimal for ~ 10 bits/key)
- Sharded: 64 partitions of $\sim 190\text{ MB}$ each across the dedup tier

```
# URL membership check (probabilistic + exact backstop)
def is_new_url(url):
    h = url_canonical(url)
    if not bloom.might_contain(h):
        bloom.add(h)
        rocks.put(h, 1)
        return True    # definitely new
    # Bloom hit: could be false positive (~1%)
    if rocks.get(h) is None:
        rocks.put(h, 1)
        return True
    return False      # already seen
```

Tunings

- **Counting Bloom**: if you ever need to remove URLs (e.g., re-crawl invalidation)
- **Cuckoo filter**: better fp rate at the same memory; supports deletion
- **Partition by host hash**: co-locates URLs from the same host for cache locality

10

Content Deduplication

SimHash & near-duplicates

URL dedup catches obvious repeats. **Content dedup** catches the long tail of mirror sites, syndication, boilerplate, and tracker-only differences. Use a **locality-sensitive hash** (SimHash for text, MinHash for sets) so that near-identical pages collapse into the same bucket.

SimHash Algorithm

```
# 64-bit SimHash over n-gram shingles of the page body
def simhash(tokens, bits=64):
    v = [0] * bits
    for tok in tokens:
        h = hash64(tok)
        for i in range(bits):
            v[i] += 1 if (h >> i) & 1 else -1
    fingerprint = 0
    for i in range(bits):
        if v[i] > 0:
            fingerprint |= (1 << i)
    return fingerprint

def is_near_duplicate(fp, store, hamming_thresh=3):
    # Index fp into 4 banded tables of 16 bits each
    for band in band_keys(fp):
        for cand in store.lookup(band):
            if popcount(fp ^ cand) <= hamming_thresh:
                return True
    store.insert(fp)
    return False
```

SimHash vs Exact Hash

Property	Exact (SHA-256)	SimHash
Catches near-dups	No (1-bit change → new hash)	Yes (Hamming distance ≤ 3)
Index size	32 B / page	8 B / page
Lookup	O(1) hash map	Banded LSH; O(1) per band
False positives	Effectively 0	~ small; tunable by Hamming threshold

Property	Exact (SHA-256)	SimHash
Use case	Byte-identical detection	Boilerplate / mirror / syndication

Pipeline Placement

- Compute fingerprint after parsing & boilerplate stripping
- Skip storage write if near-dup found (saves ~30% of bytes)
- Still record the URL → canonical-page mapping for reverse lookups

11

Storage Tiers

Hot / warm / cold

Tiered Architecture

Tier	Backend	Contents	Latency	Cost
Hot	HBase / Cassandra	Page metadata, last 7 days raw HTML for active pages	ms	~\$0.10 / GB-mo
Warm	S3 Standard (WARC files)	Compressed raw HTML, 90-day window	10-100 ms	~\$0.023 / GB-mo
Cold	S3 Glacier	Older WARC archives, 1-7 yr	min-hours (restore)	~\$0.004 / GB-mo

Page Metadata Schema (HBase)

```
-- Logical schema for page metadata
CREATE TABLE page_metadata (
  url_hash      BINARY(16) PRIMARY KEY,  -- md5(canonical_url)
  canonical_url TEXT NOT NULL,
  host          VARCHAR(253) NOT NULL,
  http_status   SMALLINT,
  content_type  VARCHAR(64),
  content_len   INT,
  simhash       BIGINT,                  -- 64-bit fingerprint
  s3_warc_key   VARCHAR(255),            -- pointer to raw bytes
  s3_warc_offset BIGINT,
  fetched_at    TIMESTAMP,
  next_refresh  TIMESTAMP,              -- frontier picks this up
  priority      TINYINT,                 -- 1=hourly..4=monthly
  outlink_count INT,
  INDEX idx_host      (host),
  INDEX idx_next_refresh (next_refresh),
  INDEX idx_simhash   (simhash)
);
```

WARC Files

- **Format:** Web ARChive (ISO 28500); standard for crawl dumps
- **Roll-over:** 1 GB per WARC file; ~33K files/day

- **Compression:** per-record gzip; ~5× compression on HTML
- **Index:** CDX index file maps url_hash → (warc_key, offset)

INSIGHT

Why HBase + S3?

HBase gives random access by url_hash with mutable metadata (refresh times, priority, fingerprints). S3 stores the bulky immutable raw HTML cheaply. Splitting hot metadata from cold body bytes is the key cost lever at petabyte scale.

12

Refresh & Recrawl Policy

Keeping the index fresh

Priority Tiers

Tier	Examples	Refresh
P1: hourly	News homepages, twitter trending, status pages	1 hour
P2: daily	Top-10% by inlinks; popular blogs	1 day
P3: weekly	Mid-tail content sites	7 days
P4: monthly	Long tail (>90% of corpus)	30 days

Adaptive Refresh

- Track **change rate** per URL: did the simhash differ on last refetch?
- If unchanged for k consecutive cycles ⇒ **downgrade** priority tier
- If changed every cycle ⇒ **upgrade** tier
- Use ETag / Last-Modified / If-None-Match to skip unchanged pages cheaply

Sitemaps & Pings

- **sitemap.xml**: trust hosts that publish one; seed frontier with it
- **RSS / Atom**: high-signal change notification for blogs
- **PubSubHubbub / WebSub**: push-based; instant notification of changes
- **Effect**: reduces wasted refetches by ~40% for cooperating hosts

TIP

BFS vs DFS

Crawl breadth-first. BFS keeps the frontier diverse and politeness-friendly (many hosts, never deep on one). DFS would hammer a single host and miss broad coverage. Add priority on top of BFS: high-importance pages jump the queue but BFS is the default ordering.

13

Scalability & Distribution

Sharding the crawler

Partitioning

- **Shard by host hash:** a single host always lands on the same frontier shard
- **Why host, not URL:** politeness state (rate, robots) is per-host
- **Re-sharding:** consistent hashing $\Rightarrow$ only $1/N$ of hosts move on scale change
- **Cross-shard links:** emit to a Kafka topic partitioned by host hash

Topology

- **Frontier shards:** ~64 partitions; each owns ~150K hosts
- **Fetcher pool:** stateless; any pod can fetch from any shard
- **Parser pool:** stateless; consumes from per-shard Kafka topic
- **Dedup tier:** sharded Bloom + RocksDB by url_hash prefix
- **Object storage:** S3 is region-global; HBase is geo-replicated

Backpressure

- **Kafka between stages:** absorb fetcher / parser rate mismatches
- **Frontier soft cap:** if back-queues exceed M URLs/host, stop accepting
- **Drop priority tail:** under sustained overload, drop P4 enqueues first
- **Auto-scale fetchers** on queue depth and per-pod CPU

INSIGHT

Stateless workers, stateful queues

Push *all* stateful concerns into the frontier and dedup layers. Keep fetchers and parsers stateless so they can be scaled, replaced, or restarted at any time. This is the same pattern as MapReduce: stateless compute, durable shuffle.

14

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
Fetcher pod crash	In-flight URLs lost	Pod liveness; missed heartbeat	K8s reschedules; URLs re-emerge from frontier WAL
Frontier shard down	Hosts on that shard pause	Replica lag alert	Promote standby; replay WAL; resume
DNS outage	Most fetches fail	DNS error rate	Failover to secondary resolver; serve from cache
Slow host (tarpit)	Workers stuck on it	Per-host p99 latency	Per-host timeout; circuit-break for 1 h
Bloom corruption	Re-fetch already-seen URLs	Disk checksum mismatch	Rebuild from RocksDB snapshot; takes hours
S3 region outage	Cannot persist new HTML	PUT error rate	Buffer in local disk; replay when S3 recovers
Spider trap (infinite URLs)	Frontier pollution	Per-host URL growth	Cap URLs/host; depth limit; pattern detection
Robots violation	Host complaints; IP ban	Webmaster contact; abuse@	Revalidate robots cache; honor Retry-After

Idempotency

- **Frontier dequeue:** at-least-once; URLs may be fetched twice on crash
- **Storage writes:** idempotent on (url_hash, fetched_at)
- **Dedup checks:** safe under retry; Bloom + RocksDB are monotonic
- **Effect:** end-to-end at-least-once; duplicates collapsed by url_hash key

WARNING

Spider Traps

Calendars, faceted-search pages, session IDs in URLs, and infinite redirect chains can generate unbounded URLs from a single host. Defend with **per-host URL caps**, **depth limits**, **query-param fingerprinting**, and **refresh demotion** on near-duplicate detection.

15

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Traversal order	BFS + priority	Diverse coverage, polite. DFS would hammer hosts and miss breadth.
Politeness scope	Per-host (with per-IP cap)	Matches robots.txt scope. Pure per-IP is friendlier to shared hosting but loses cleanliness.
URL dedup	Bloom + RocksDB	Fast in-memory check, exact backstop. Pure RocksDB is too slow; pure Bloom has false positives.
Content dedup	SimHash (LSH)	Catches near-duplicates at 8 B/page. Exact hash misses 30%+ of mirror content.
Frontier	Mercator two-level queue	Decouples priority from politeness. A single FIFO queue cannot do both.
Storage split	HBase metadata + S3 WARC	Cheap bulk + fast metadata. All-HBase is too expensive; all-S3 has no random access.
Refresh policy	Adaptive (4 tiers)	Spend budget on changing pages. Fixed-period wastes ~40% on static content.
JS rendering	Phase 2 only	Headless Chrome is ~50× more expensive per fetch. Worth it only for SPA-heavy hosts.
Storage tier	S3 Glacier for > 90 d	Cuts cost ~6×. Restore latency is hours, but cold archives are rarely queried.
Fresh vs polite	Politeness wins	Never break robots.txt. Freshness loss recovered with sitemaps and PubSubHubbub.

16

Interview Playbook

How to present this

Web crawler is a classic SD interview because it forces you to balance five orthogonal concerns: scale, politeness, dedup, freshness, and storage cost. Lead with the loop, anchor the math, and defend trade-offs.

45-Minute Interview Outline

1. **Intro (2 min):** 10B URLs, 30-day refresh, polite, near-dup detection
2. **Capacity (5 min):** 3,860 fetches/sec, 33 TB/day raw, 750 TB/yr stored
3. **High-level arch (4 min):** frontier → DNS → fetch → parse → {links, store, index}
4. **Frontier deep dive (8 min):** Mercator front+back queues; politeness heap
5. **Dedup deep dive (8 min):** Bloom for URLs, SimHash for content
6. **Storage (5 min):** HBase metadata + S3 WARC + Glacier cold
7. **Refresh (4 min):** 4 priority tiers + adaptive demotion
8. **Failures (4 min):** spider traps, slow hosts, robots violations
9. **Wrap (5 min):** trade-offs and what you'd build next (JS rendering)

Key Talking Points

- “3,860 fetches/sec sustained, ~8K peak” — capacity math, not magic
- “Mercator two-level queue” — cite the canonical design by name
- “Politeness is per-host with per-IP cap” — nuanced, correct
- “Bloom filter at 10 bits/key → 12 GB for 10B URLs” — exact memory math
- “SimHash with Hamming threshold 3” — concrete near-dup parameter
- “HBase + S3 + Glacier” — cost-aware tiered storage
- “Adaptive refresh on simhash drift” — freshness without waste
- “Idempotent at-least-once” — recovery story is correct, not hand-wavy

Common Follow-ups & Answers

- **Q: How do you avoid hammering one host?** A: Per-host back-queue + min-heap on next_fetch_at; honour robots Crawl-Delay, default 1 req/sec.
- **Q: Why Bloom + RocksDB instead of just one?** A: Bloom skips 99% of disk reads; RocksDB resolves the ~1% Bloom false-positives. Pure Bloom would re-fetch 1% of URLs forever.
- **Q: What about JavaScript-rendered pages?** A: Phase 2: headless Chrome pool, ~50× more expensive; reserve for SPA-heavy hosts identified by content signals.

- **Q: How do you detect spider traps?** A: Per-host URL growth metric, depth limits, query-param fingerprint, simhash-driven refresh demotion.
- **Q: What if the frontier shard crashes?** A: WAL on disk; standby promotes and replays. URLs may be fetched twice (at-least-once); idempotent storage by url_hash.
- **Q: Why not Cassandra over HBase?** A: Either works; HBase has stronger consistency on row updates (refresh timestamps), Cassandra wins on multi-region writes. Pick based on existing stack.

KEY

Memorize These Numbers

10B URLs · 30-day refresh · **3,860 fetches/sec** sustained, **~8K peak** · 100 KB avg page · **33 TB/day** raw ⇒ **~750 TB/year** stored · **~12 GB** Bloom filter · **1 req/sec/host** politeness · **~1,600 fetcher workers**.